# Smart Inventory & Warehouse Automation Platform

## 1.Problem Analysis

### Business Context

The company runs several warehouses and retail stores, and right now inventory is tracked mostly by hand across spreadsheets. That creates blind spots. Fast selling items run out before anyone notices, slow movers pile up and tie up cash, and purchase orders often go out too late to prevent a shortage. Warehouse managers end up spending hours every week just checking stock levels, chasing suppliers for updates, and manually matching deliveries against what was ordered, time that could go toward more useful work.

### Stakeholders

Warehouse managers are the ones who feel this most directly. They need to know, in real time, what is running low and whether a delivery has actually arrived, without having to go dig through a spreadsheet themselves. The procurement team needs purchase requests raised automatically and sent through a proper approval process instead of ad hoc emails. Finance and other approvers need a say before any large purchase gets committed. Suppliers need purchase orders that are timely and accurate, since confusion on their end just adds more delay. Leadership wants periodic insight into how healthy the inventory situation is overall, and how suppliers are performing. And whoever owns this platform technically needs it to be reliable and easy to trace when something goes wrong.

### Pain Points

Stock checks happen too infrequently to catch problems early, so shortages are often discovered only once it is already too late to fix quietly. Purchase orders tend to be reactive rather than planned ahead, which causes delivery delays and, at times, lost sales. There is no consistent record of who approved what spending and when, since a lot of these decisions currently happen informally over email or phone. Matching deliveries against purchase orders is done manually, which invites mistakes. There is no single place to see how inventory is trending or how suppliers are performing, so putting together that picture takes real manual effort. And when something in the process fails, there is no audit trail to help figure out what happened or why.

### Objectives

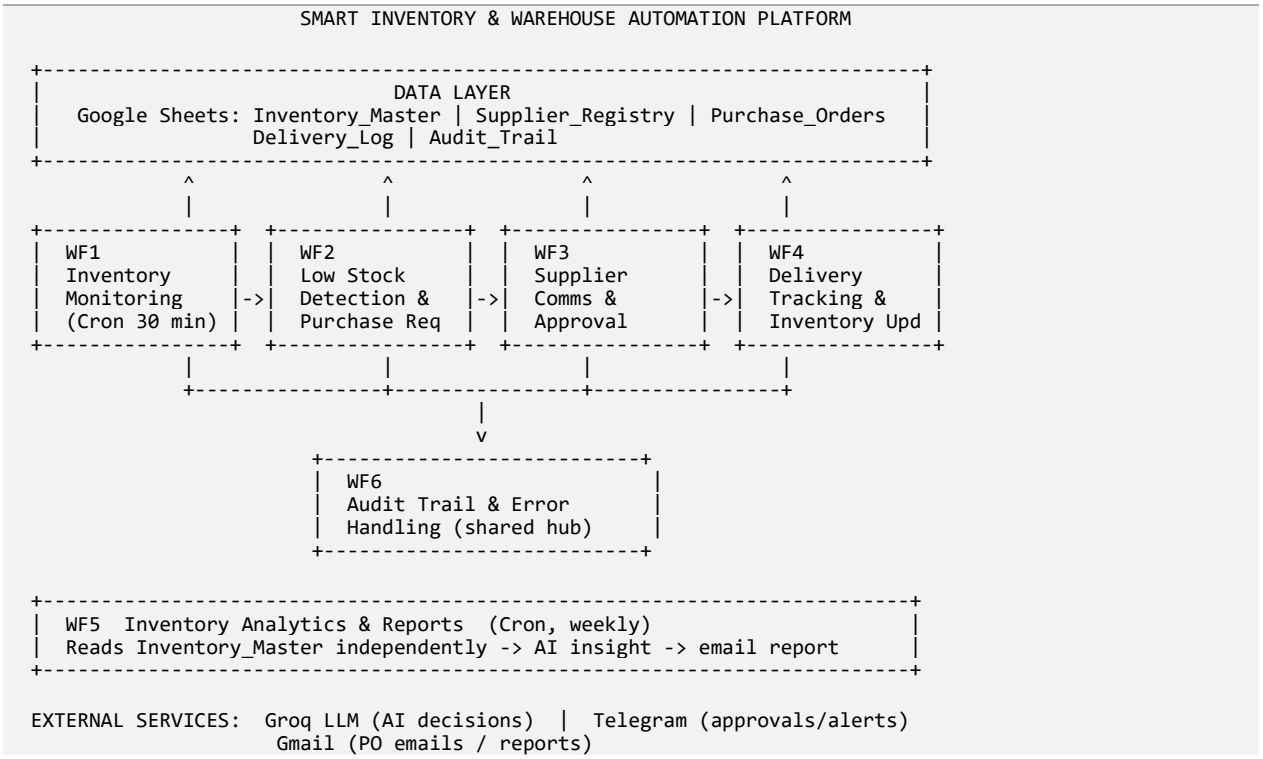
The platform should watch inventory levels continuously across every product and warehouse without needing a person to check manually. It should catch low stock situations early and use AI to suggest a sensible reorder quantity rather than a guess. It should route purchase orders for human approval whenever the spend crosses a set threshold, so nothing large goes out unchecked. It should automatically reconcile deliveries against the original purchase orders and update stock the moment goods arrive. It should generate recurring reports with AI written insights so leadership gets a clear picture without anyone compiling it by hand. And it should

keep a full audit trail of every automated action and every error, so the whole system stays accountable and easy to troubleshoot.

# 2.Workflow Architecture

## Overall System Architecture

The platform is built as six independent n8n workflows that all read and write a shared Google Sheet acting as the system of record. Four workflows form the core operational pipeline, one runs a weekly analytics cycle, and one acts as a shared audit and error handling hub that the others report into.




## Workflow Interaction Diagram

Workflows communicate with each other over HTTP webhooks hosted on the same n8n instance. Each workflow exposes a webhook path that the previous workflow in the chain calls once its own step is complete.

```
WF1 --(HTTP POST /low-stock-trigger)--> WF2
WF2 --(HTTP POST /po-approval)---------> WF3
WF3 --(Wait node + resumeUrl, human click)--> resumes WF3 itself
WF4 <--(external delivery event, POST /delivery-received)-- Supplier/warehouse system
WF1 --(HTTP POST /audit-log)-----------> WF6
WF2, WF3, WF4 --(errorWorkflow setting)--> WF6 (Error Trigger)
WF5 -- runs independently, no inter-workflow call

All workflows read/write the same Google Sheet (shared state),
so the sheet itself is also an implicit integration point,
not just the HTTP calls between workflows.
```

### Event Flow Between Workflows

The sequence below traces a single item from the moment it is detected as low stock through to the delivery being received and the stock being updated, plus the parallel weekly reporting cycle and the always on audit logging.

```
STEP 1  Cron fires (every 30 min)
        WF1 reads Inventory_Master
        WF1 filters items where current_stock <= reorder_point
        IF none low -> WF1 logs "SCAN" event to WF6 and stops
        IF low stock found -> WF1 POSTs items[] to WF2, then logs to WF6

STEP 2  WF2 webhook receives items[]
        For each item: WF2 looks up Supplier_Registry
        WF2 asks Groq LLM for recommended order_quantity + reasoning
        WF2 builds a PO record (po_id, qty, cost, status = "Pending Approval")
        WF2 appends PO to Purchase_Orders sheet
        WF2 POSTs the PO to WF3

STEP 3  WF3 webhook receives PO
        IF estimated_cost > $5000:
            WF3 sends Telegram approval request (Approve / Reject links)
            WF3 pauses on a Wait node until a manager clicks a link
            On resume: IF approved -> update status to "Approved", email supplier
                       IF rejected -> update status to "Rejected", stop
        ELSE (<= $5000):
            Auto-approved -> status set to "Approved" -> email sent to supplier

STEP 4  Supplier fulfills -> delivery event fires (external system) POST to WF4
        WF4 looks up the original PO by po_id
        WF4 checks delivered quantity against ordered quantity
        WF4 fetches current stock, adds delivered quantity
        WF4 updates Inventory_Master, appends to Delivery_Log
        WF4 sends a Telegram delivery confirmation

STEP 5 (parallel, weekly)
        WF5 cron fires every week
        WF5 reads Inventory_Master, computes low stock / overstock counts
        WF5 asks Groq LLM to write a short analyst-style report
        WF5 emails the report to leadership

THROUGHOUT
        WF6 listens on /audit-log for explicit events (e.g. WF1 scans)
        WF6 also catches runtime errors from any workflow configured
        to use it as their error workflow, logs to Audit_Trail,
        and sends a Telegram alert for CRITICAL severity errors
```

# 3.Implementation

The platform is implemented as six independent, individually deployable n8n workflows, exceeding the minimum of four to six required. Together they total 42 nodes, well above the 20 to 25 node minimum. Each workflow owns a single responsibility and communicates with the others over HTTP webhooks, with Google Sheets acting as the shared system of record.

## Workflow Summary

| ID | Workflow | Trigger | Node count |
|---|---|---|---|
| WF1 | Inventory Monitoring & Stock Updates | Cron (every 30 min) | 6 |
| WF2 | Low Stock Detection & Purchase Request | Webhook (low-stock-trigger) | 8 |
| WF3 | Supplier Communication & Approval | Webhook (po-approval) | 8 |
| WF4 | Delivery Tracking & Inventory Update | Webhook (delivery-received) | 7 |
| WF5 | Inventory Analytics & Reports | Cron (weekly) | 6 |
| WF6 | Audit Trail & Error Handling | Webhook (audit-log) + Error Trigger | 7 |
| | | **Total nodes** | **42** |

## Node Breakdown by Workflow

**WF1: Inventory Monitoring & Stock Updates** (6 nodes)
Cron Trigger, Read Inventory_Master, Check Low Stock, Any low stock?, Trigger WF2, Log Scan

**WF2: Low Stock Detection & Purchase Request** (8 nodes)
Webhook Trigger, Item Lists, Get Supplier Info, Groq Chat Model, Basic LLM Chain, Create PO Record, Append to Purchase_Orders, Trigger WF3

**WF3: Supplier Communication & Approval** (8 nodes)
Webhook Trigger, IF > $5000, Telegram Request Approval, Wait, Is Approved?, Update PO Status (Approved), Update PO Status (Rejected), Send PO Email

**WF4: Delivery Tracking & Inventory Update** (7 nodes)
Webhook Trigger, Lookup PO, Qty Matches?, Get Current Stock, Update Inventory, Append to Deliveries, Telegram Notification

**WF5: Inventory Analytics & Reports** (6 nodes)
Cron Trigger (Weekly), Read Inventory, Aggregate Metrics, AI Generate Insights, Basic LLM Chain, Email Report

**WF6: Audit Trail & Error Handling** (7 nodes)
Webhook Trigger, Format Log Entry, Append to Audit_Trail, Error Trigger, Format Error Details, Append Error to Audit_Trail, Telegram Alert

### Integrations Used

Google Sheets is used as the primary datastore across all six workflows, covering the Inventory_Master, Supplier_Registry, Purchase_Orders, Delivery_Log, and Audit_Trail sheets. Groq, accessed through the LangChain nodes, powers the AI decision points in WF2 and WF5. Telegram is used for human approval requests and alerts in WF3, WF4, and WF6. Gmail is used to send purchase order emails to suppliers in WF3 and the weekly analytics report in WF5. Custom Code nodes handle data transformation, filtering, and formatting throughout, and native If nodes handle all conditional branching.

## 4.Advanced Features

All seven advanced feature categories from the assignment brief are represented in the platform, several of them across more than one workflow. The table below gives a quick reference, followed by a short explanation of how each one is actually implemented.

### Coverage Summary

| Feature | Implemented in |
|---|---|
| AI-powered decision making | WF2, WF5 |
| Human approval step(s) | WF3 |
| Error handling and retry logic | WF6, plus HTTP nodes across WF1 to WF4 |
| Logging and audit trail | WF6, called from WF1 |
| Scheduled workflows (Cron) | WF1, WF5 |
| Webhook-triggered workflows | WF2, WF3, WF4, WF6 |
| Conditional branching and loops | WF1, WF3, WF4 |

### How Each Feature Is Implemented

#### AI-powered decision making

WF2 calls a Groq-hosted LLM through a LangChain node to recommend an optimal reorder quantity for each low-stock item, using current stock, reorder point, max stock level, and the supplier's average delivery time as inputs, along with a written justification. WF5 calls the same

model weekly to turn raw stock numbers into a short, readable analyst-style summary for leadership.

**Human approval step(s)**
Purchase orders above 5000 dollars are routed to a manager over Telegram with Approve and Reject links before anything is sent to the supplier. The workflow pauses on a Wait node until the manager responds, then resumes and updates the purchase order status accordingly.

**Error handling and retry logic**
WF6 includes a dedicated Error Trigger that any other workflow can be configured to call when a run fails. It captures the failing workflow's name and error details, logs them to the Audit_Trail sheet, and sends a Telegram alert for critical severity failures. The HTTP Request nodes used for inter-workflow calls can additionally be configured with n8n's built-in retry-on-fail and continue-on-fail settings for transient network issues.

**Logging and audit trail**
WF6 acts as a shared logging hub. WF1 posts scan summaries to it after every run, and it can accept events from any other workflow the same way. Every entry is timestamped, tagged with an event ID, source workflow, event type, and severity, and appended to the Audit_Trail sheet, giving a single place to trace what happened across the whole platform.

**Scheduled workflows (Cron)**
WF1 runs on a Cron trigger every 30 minutes to continuously scan inventory levels. WF5 runs on a weekly Cron trigger to generate the analytics report, so reporting happens on a predictable cadence without anyone needing to run it manually.

**Webhook-triggered workflows**
WF2, WF3, WF4, and WF6 are all triggered by inbound webhooks rather than a schedule, which lets them react immediately to events: a low stock detection from WF1, a purchase order ready for approval, a delivery arriving, or an audit event being raised, instead of waiting for the next polling cycle.

**Conditional branching and loops**
WF1 branches on whether any items are below their reorder point before deciding whether to trigger WF2. WF3 branches twice, once on whether the order value exceeds the 5000 dollar approval threshold, and again on whether the manager approved or rejected the request. WF4 branches on whether the delivered quantity matches the ordered quantity. WF2 iterates over the full list of low-stock items coming from WF1, running the supplier lookup and AI recommendation once per item.